香港教育大學
The Education University of Hong Kong

MSc(AIET) 2025-26

MTH6184 Data Mining in STEM Education

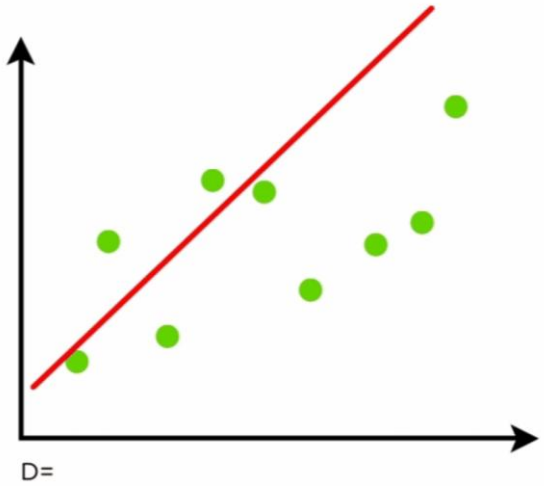
Chapter 3. Regression and Decision Tree

Instructor: Philip Yu

Chapter Outcomes

After completing this chapter, you should:

- Understand how to build linear and logistic regression models
- Understand ridge and lasso regression for parameter regularization
- Understand how a decision tree works
- Understand how to build a decision tree via recursive partitioning
- Know how to find a reasonable size of a tree



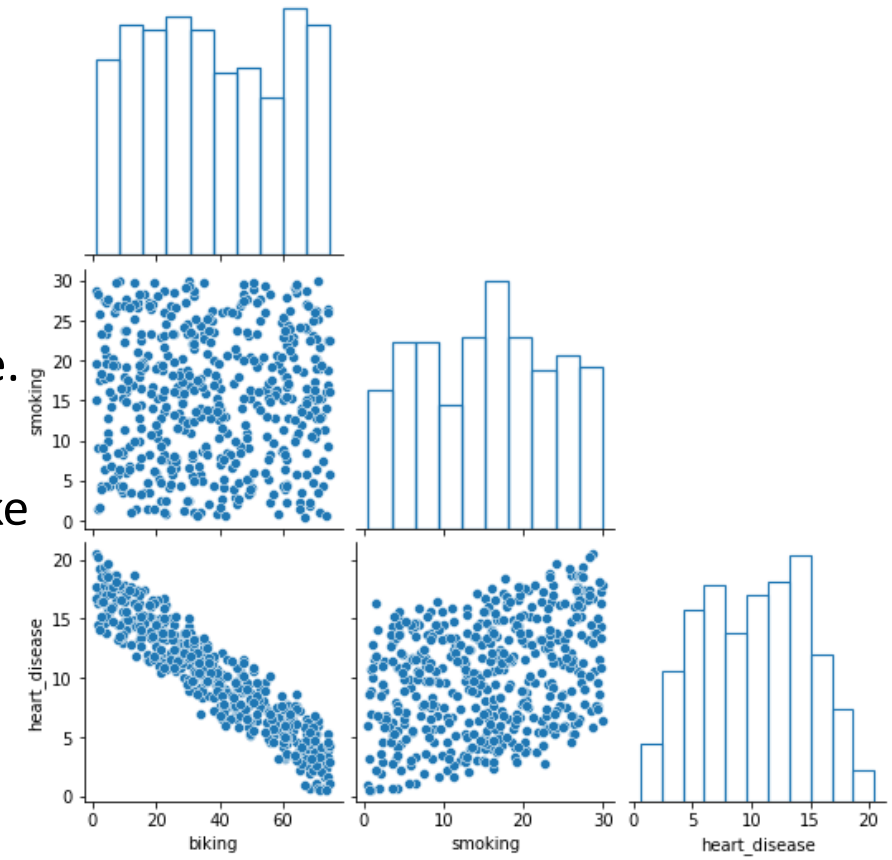
Public Health Example

- Consider a public health researcher who is interested in social factors that influence heart disease. He/she surveyed 498 towns and gathered data on three variables:

  (1) the percentage of people in each town who smoke,

  (2) the percentage of people in each town who bike to work, and

  (3) the percentage of people in each town who have heart disease.
- In this example, (3) is the output (or target) variable while (1) and (2) input variables. Since all variables are quantitative, we can make use of linear regression model to model their linear relationship.
- Before we make any computation on the linear regression model, the matrix of scatter plots can help to obtain their preliminary pairwise relationship.
- Can we observe any relationship in the pairs (3,1) and (3,2)?



Linear Regression

```
sklearn.linear_model.LinearRegression:
fit_intercept = True | False

.fit (fit linear model)
.predict (predict using the linear model)
```

- A linear regression model is used to model the linear relationship between one output variable Y and two or more input variables X's:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_k x_{ik} + \varepsilon_i \quad \text{for } i = 1,2,\ldots,n,$$

where y_i = i^{th} value of observed output variable

x_{ij} = i^{th} value of the observed input variable X_j for $j = 1,2,\ldots,k$,

β_0 = intercept, the effect on the output variable Y when all $X_j = 0$ for $j = 1,2,\ldots,k$,

β_j = marginal effect of X_j on the output variable Y,

ε_i = random error with zero mean and constant variance σ^2

- The mean of output variable Y can be written as

$$E(Y_i) = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_k x_{ik} \quad \text{for } i = 1,2,\ldots,n$$

- Interpret β_j: If X_j increases by 1 unit with the other X_i's fixed, then Y is expected to increase by β_j units.

Linear Regression: Estimation

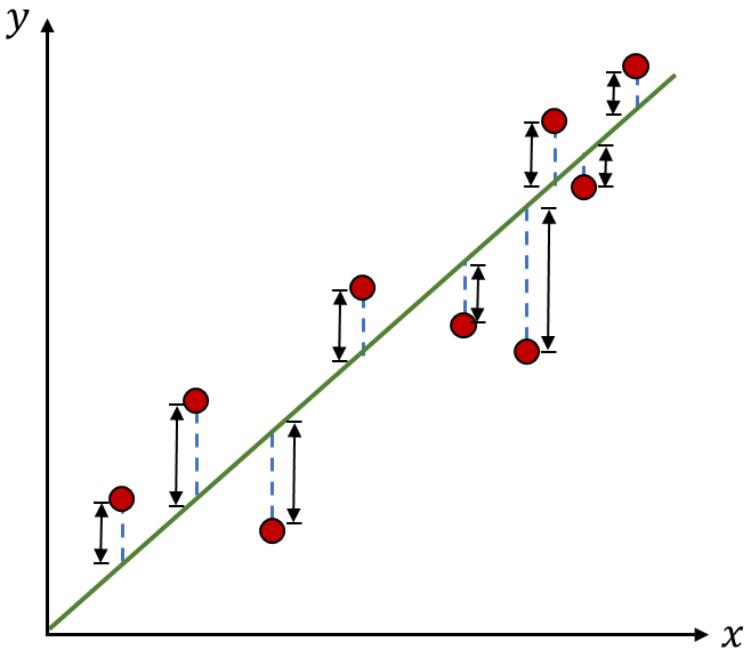
- To estimate the regression parameters $(\beta_0, \beta_1, \dots, \beta_k)$, the method of least squares is used.
- Simply speaking, we are going to choose the values of $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_k$ (estimates of $\beta_0, \beta_1, \dots, \beta_k$) which minimizes the sum of squared errors (i.e., error = $y_i - \hat{y}_i$):

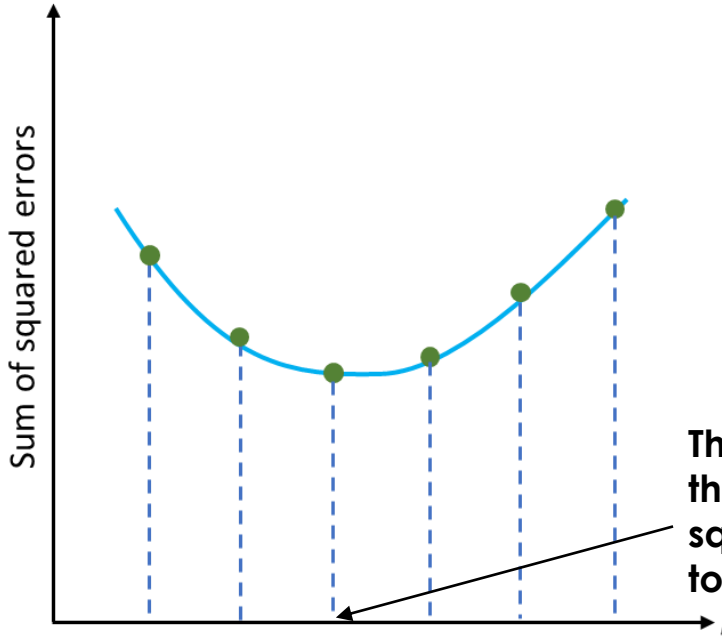
$$\hat{\beta}_{\boldsymbol{LSE}} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{k} x_{ij}\beta_j \right)^2 \right\}$$

- The fitted value of output variable: $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \cdots + \hat{\beta}_k x_{ik}$.

Reminder:

Before fitting the model, we need to create dummies for the categorical features and impute missing values (if any).





This rotation of regression line on the left is one with the "least squares", so it will be the one fit to the data.

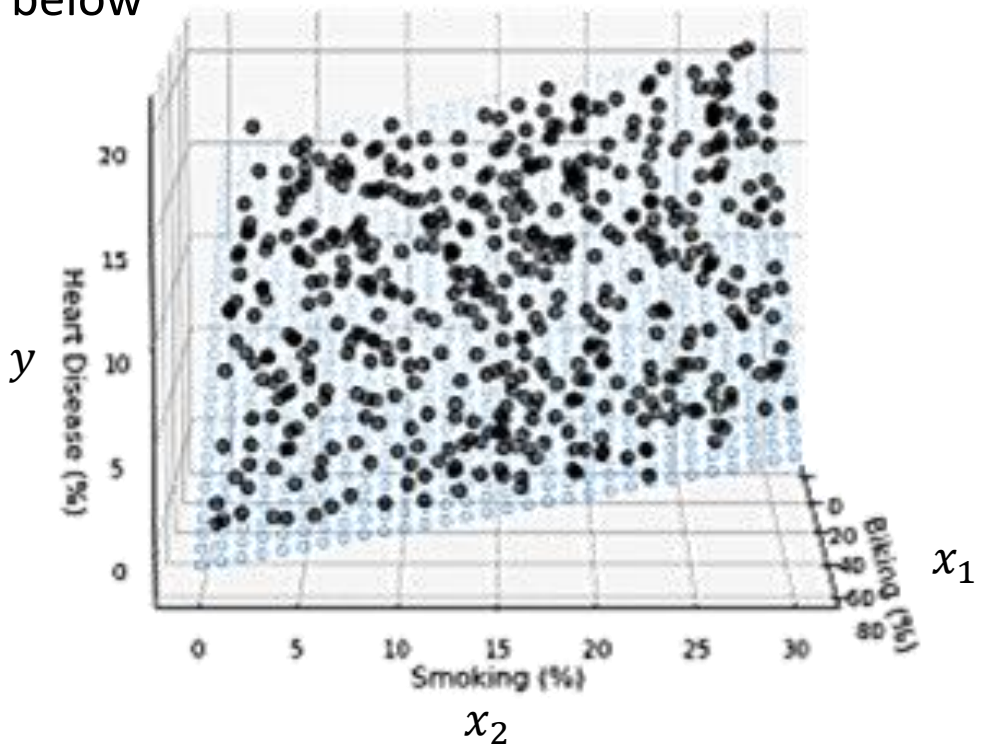
Linear Regression: Results

- Return to the real example, the fitted linear relationship of output variable and input variables in this example is

$$\hat{y}_i = \underset{0.080}{14.9847} - \underset{0.001}{0.2001}x_1 + \underset{0.004}{0.1783}x_2,$$

where 14.9847, -0.2001 and 0.1783 are the estimated coefficients $\hat{\beta}_0, \hat{\beta}_1$ and $\hat{\beta}_2$ respectively and the values below them are their standard errors.

- The fitted result is visualized below



Linear Regression

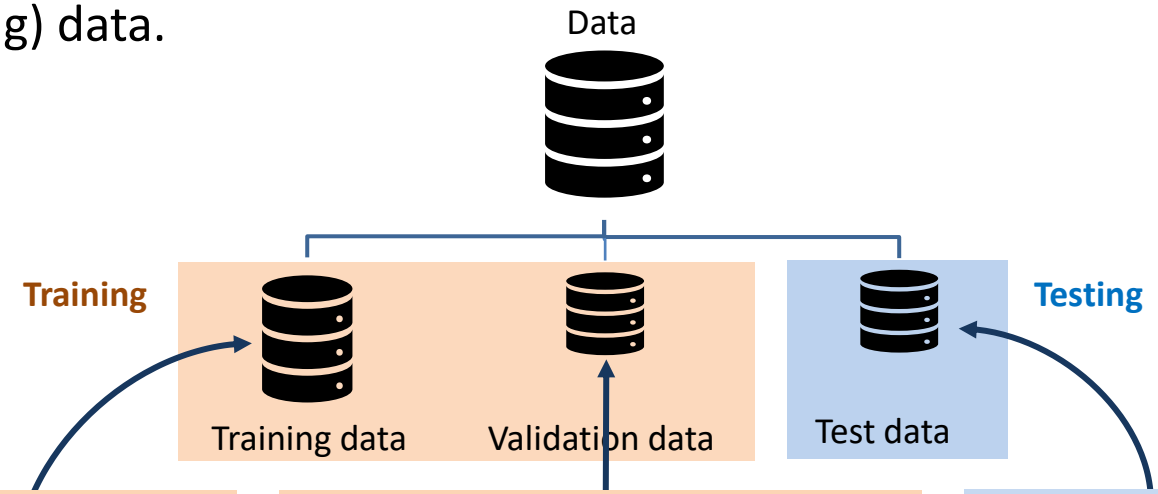
- To measure the overall goodness of fit, the measure R^2 is computed by

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

- $0 \leq R^2 \leq 1$. The larger the R^2, the smaller sum of squared errors (SSE) $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ and the better the model fits the (training) data.

Holdout method

Eg. Randomly select 70% for training and 30% for testing



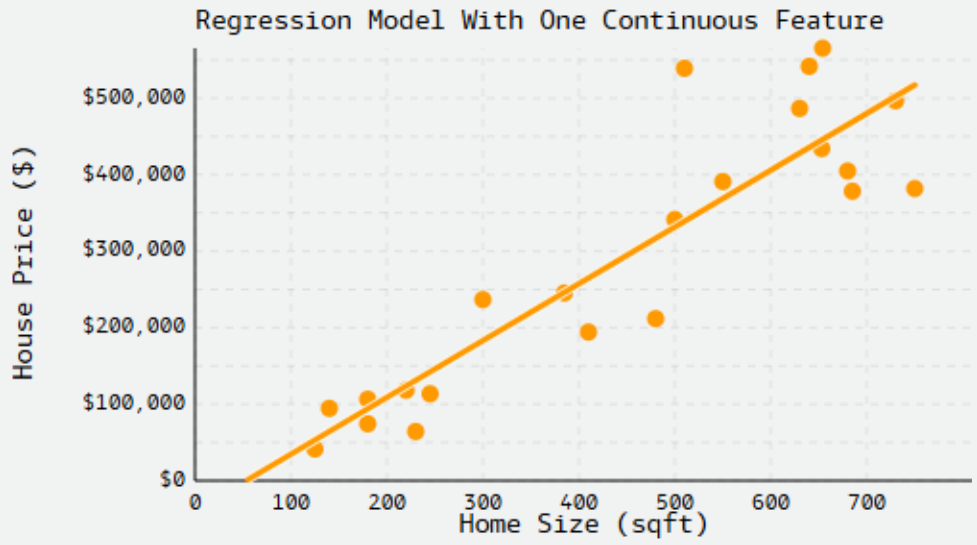
The training data is used to fit the model. During training, the parameters (or weights) of the model are adjusted by an optimization algorithm. The goal is to minimize the training loss of the model.

The validation data must be separated from the training data. It is used to tune the hyperparameters of the model and to prevent overfitting. The validation process takes place during training.

The test data must be separated from the training and validation data. It is used to evaluate the accuracy of the trained and validated model. The goal is to measure the generalization of the model.

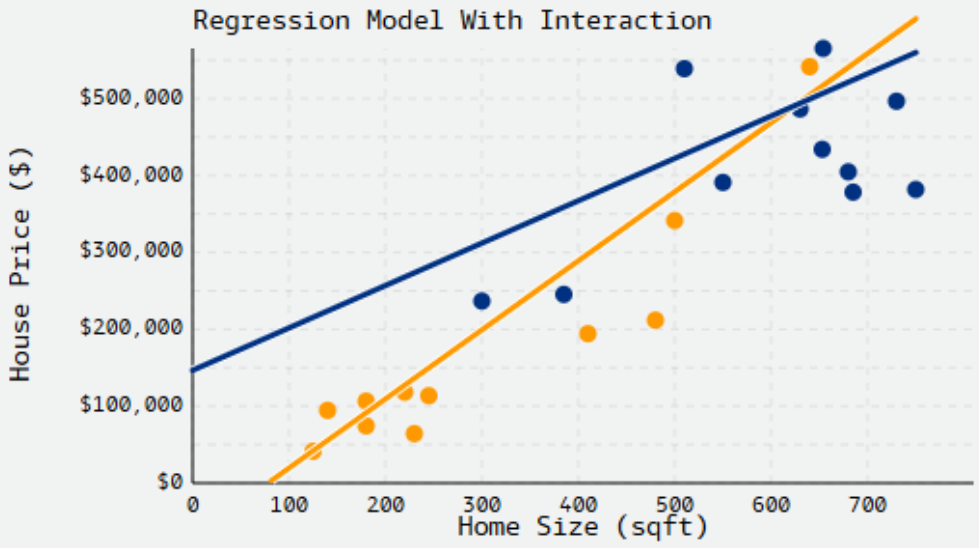
Interpreting Regression Models

A Regression Model With One Continuous Feature



Example:
house price $= -39591 + 742 * sqft$

A Regression Model With Interaction Terms



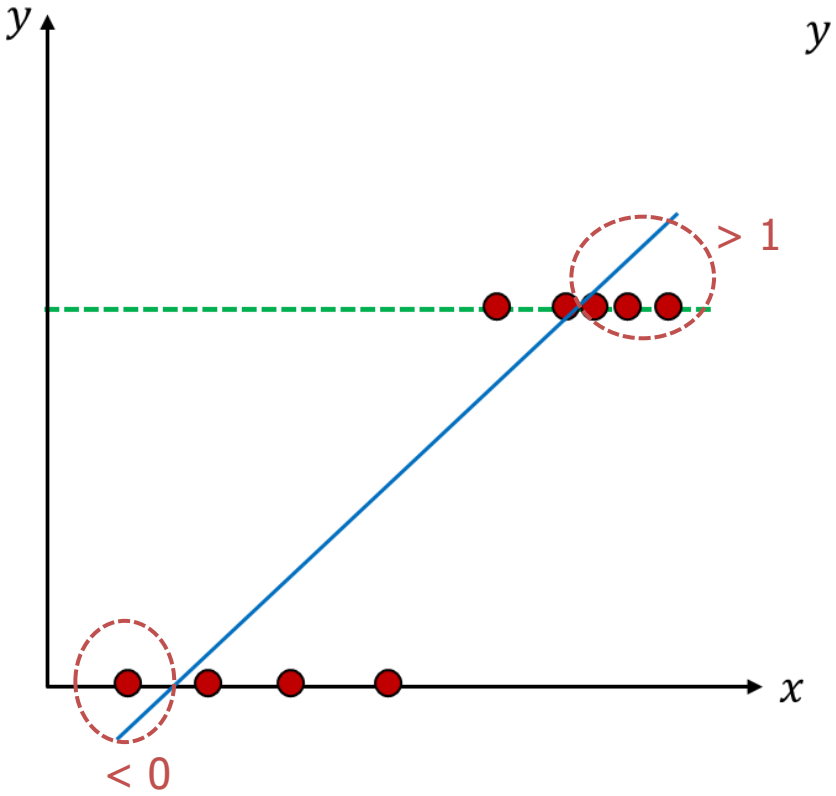
Example:
house price $= -70296 + 899 * sqft + 217111 * pool - 347 * (sqft : pool)$

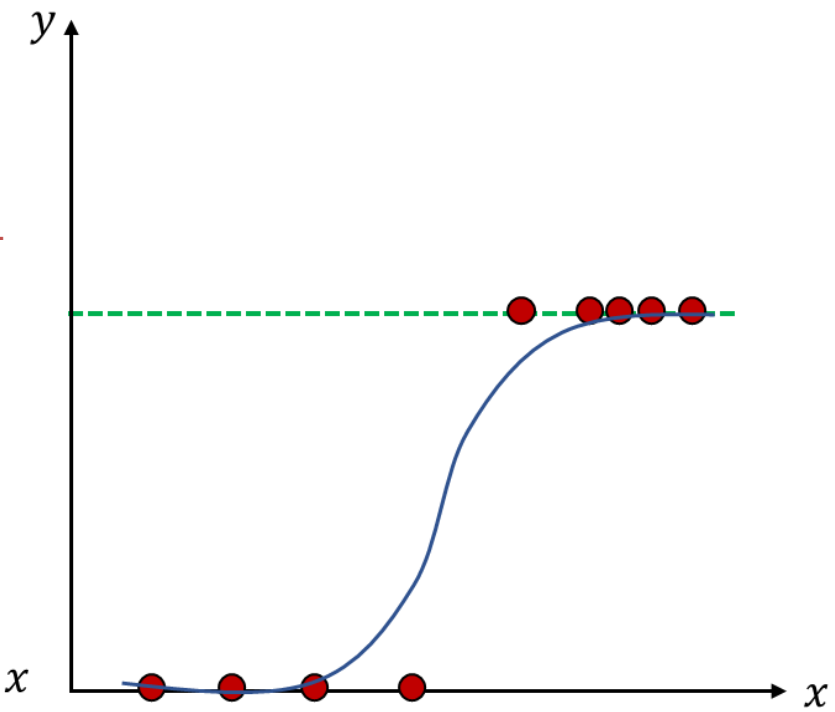
Class Activity 1:
Design a STEM Activity

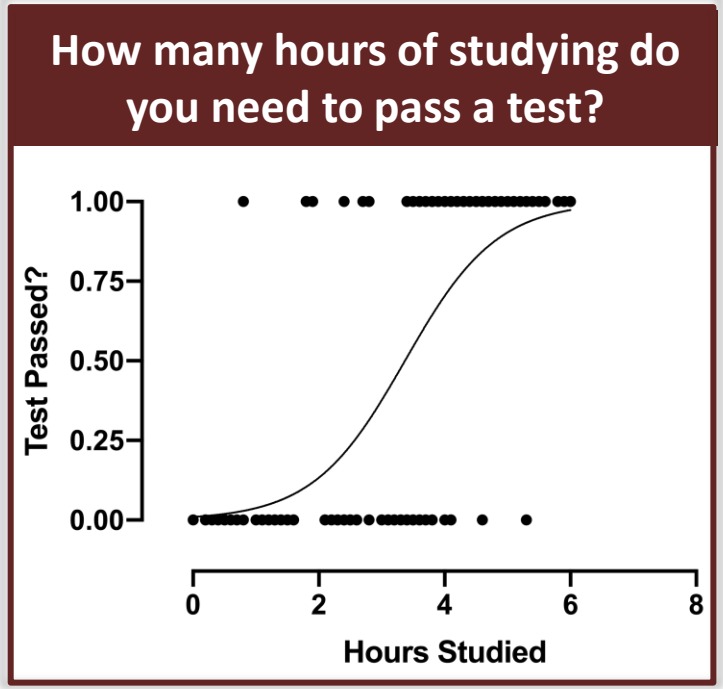
padlet.com/plhyu/lecture3

Binary Logistic Regression

- ✓ Actually, binary logistic regression is similar to linear regression.
- ✓ Their major difference is the binary output variable which produces the values of 1 or 0.
- ✓ Linear fit of binary output variable on input variable will generate undesirable prediction of Y.







- ✓ Instead of a straight line, a s-shape curve is considered to fit the data points.

Binary Logistic Regression

```
sklearn.linear_model.LogisticRegression:
fit_intercept = True

.fit (fit linear model)
.predict (predict class label)
.predict_proba (predict probability)
```

- ✓ Note that Y is a binary variable:

$$Y_i = \begin{cases} 1 & \text{event for case } i \\ 0 & \text{non-event for case } i \end{cases}$$

- ✓ Let $p_i = \Pr(Y_i = 1)$. A binary logistic regression model is defined by

$$\text{logit}(p_i) \equiv \ln \frac{p_i}{1 - p_i} = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_k X_{ik} = \boldsymbol{X}_i \boldsymbol{\beta}$$

- ✓ The term $\ln[p_i/(1 - p_i)]$ is called the log-odds which compares $\Pr(Y_i = 1)$ and $\Pr(Y_i = 0)$.
- ✓ Now, the coefficients $\beta_j (j = 1,2, \ldots, k)$ cannot be interpreted as the marginal effect of X_j on Y, but the marginal effects on the log-odds.
- ✓ When the unknown β's are replaced by their estimates $\hat{\beta}$'s, the estimated log-odds and hence predicted probability $\hat{p}$ can be expressed as

$$\log \frac{\hat{p}_i}{1 - \hat{p}_i} = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \hat{\beta}_2 X_{i2} + \cdots + \hat{\beta}_k X_{ik}$$

$$\Rightarrow \hat{p}_i = \widehat{\Pr}(Y_i = 1) = \frac{1}{1 + \exp(-(\hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \hat{\beta}_2 X_{i2} + \cdots + \hat{\beta}_k X_{ik}))}$$

Question:
How to predict Y_i from $\hat{p}_i$?

Binary Logistic Regression: Odds Ratio

- ✓ Since the odds of $Y = 1$ is computed by

$$\frac{p}{1-p} = \exp(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k),$$

the odds ratio for input X_j is

$$\frac{\exp\left(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_j (X_j + 1) + \cdots + \beta_k X_k\right)}{\exp\left(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_j X_j + \cdots + \beta_k X_k\right)} = \exp(\beta_j) \geq 0.$$

- ✓ If odds ratio $= 1$, no association between Y and X_j.
- ✓ Odds ratio is viewed as a relative risk measure.
- ✓ Eg. Confirmed COVID-19 cases by age group:

	Adult (X=0)	Elderly (X=1)	Total
Death (Y=1)	100	200	300
Survival (Y=0)	800	400	1200
Total	900	600	1500

  - ✓ Odds of dying from COVID-19 = 300/1200 = 0.25
  - ✓ Odds for Elderly (X=1) = 200/400 = 0.5
  - ✓ Odds for Adult (X=0) = 100/800 = 0.125
  - ✓ Odds ratio of dying from COVID-19 for elderly vs adult is 0.5/0.125 = 4.

Logistic Regression: General Case

Binary logistic regression:

- Y is a binary variable with $p_i = \Pr(Y_i = 1)$:

$$\text{logit}(p_i) \equiv \ln\frac{p_i}{1-p_i} = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_k X_{ik} = \boldsymbol{X}_i\boldsymbol{\beta} \quad \text{OR} \quad p_i = \frac{e^{\boldsymbol{X}_i\boldsymbol{\beta}}}{1+e^{\boldsymbol{X}_i\boldsymbol{\beta}}} = \frac{1}{1+e^{-\boldsymbol{X}_i\boldsymbol{\beta}}}$$

(Multinomial/Multiclass) logistic regression:

```
sklearn.linear_model.LogisticRegression:
multi_class = 'multinomial'
```

- Y is a categorical/multiclass variable with M classes:
- Define M binary variables $(Y_{i1}, \dots, Y_{iM})$ such that $Y_{im} = 1$ if $Y_i = m$, and $Y_{im} = 0$ otherwise.

$$(Y_{i1}, \dots, Y_{iM}) \sim \text{Mult}(1, p_{i1}, \dots, p_{iM}) \text{ with } p_{im} = \frac{e^{\boldsymbol{X}_i\boldsymbol{\beta}_m}}{\sum_{j=1}^{M} e^{\boldsymbol{X}_i\boldsymbol{\beta}_j}}, m = 1, \dots, M$$

- Loss function for obs i is the cross-entropy (or log-loss): $-\sum_{m=1}^{M} Y_{im} \ln p_{im}$
- When $M = 2$, the cross-entropy for obs i is: $-(Y_i \ln p_i + (1 - Y_i)\ln(1 - p_i))$
- The maximum likelihood method is used to estimate the β's, i.e., to maximize the likelihood function or equivalently to minimize the total cross-entropy loss.

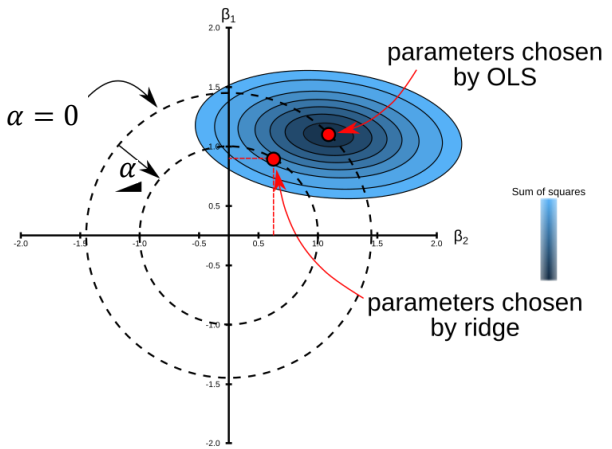
Ridge and LASSO Regression

```
sklearn.linear_model.Ridge(alpha=1.0)
sklearn.linear_model.RidgeCV(alphas=(0.1,1.0,10.0))
sklearn.linear_model.Lasso(alpha=1.0)
sklearn.linear_model.LogisticRegression (penalty =
'l2'|'l1'|'None', C=1.0)
sklearn.linear_model.LogisticRegressionCV (penalty
= 'l2'|'l1', Cs=10, cv=integer)
```

C: Inverse of regularization strength

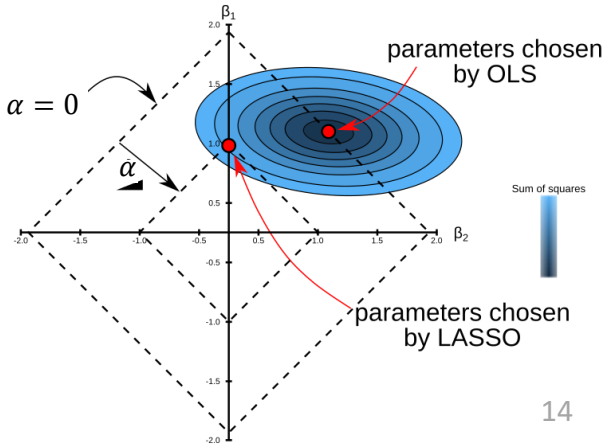
- In estimating the parameters of a linear / logistic regression model, one may want to regularize the process of parameter searching in order to
  - solve the ill-posed problem (multicollinearity problem)
  - avoid overfitting
- Ridge regression (ℓ_2 regularization):

$$\hat{\beta}^{ridge} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{k} x_{ij}\beta_j \right)^2 + \alpha \overbrace{\sum_{j=1}^{k} \beta_j^2}^{\text{penalty term}} \right\}$$



- LASSO (Least Absolute Shrinkage and Selection Operator) (ℓ_1 regularization):

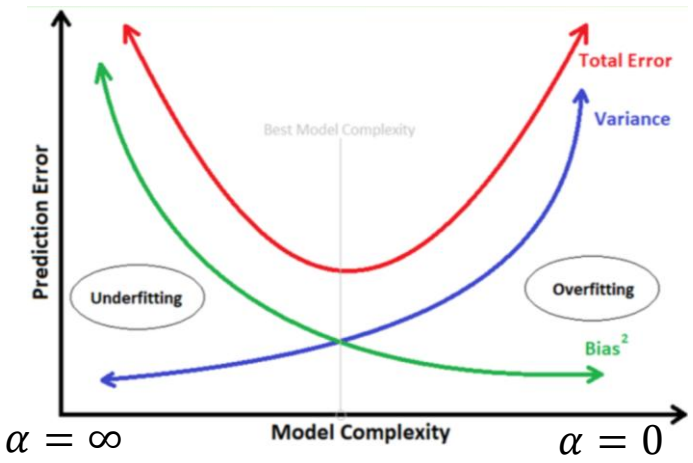
$$\hat{\beta}^{lasso} = \arg\min_{\beta} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{k} x_{ij}\beta_j \right)^2 + \alpha \sum_{j=1}^{k} |\beta_j| \right\}$$

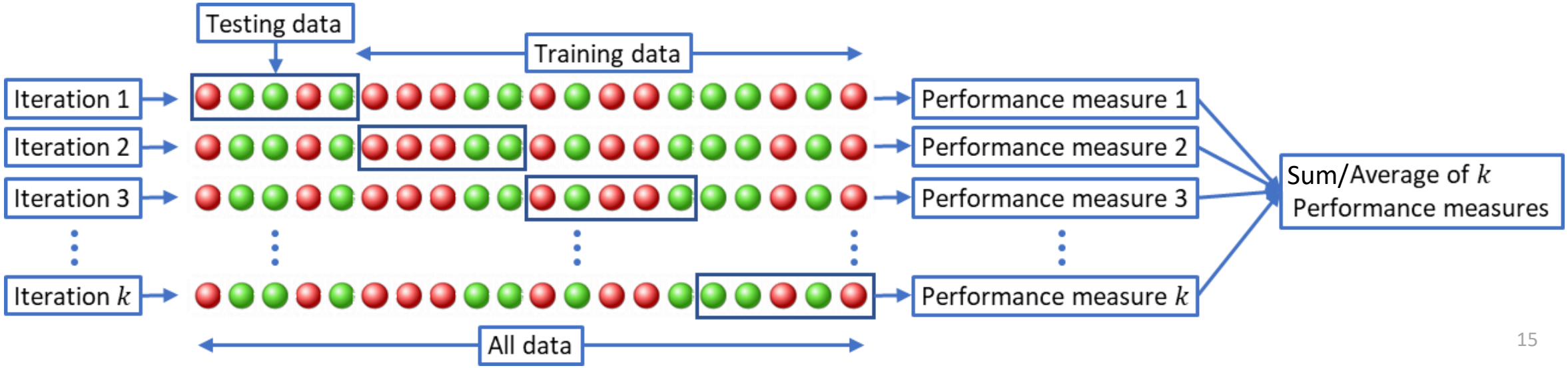


- Typically, α is chosen by cross-validation method to to achieve the best performance.

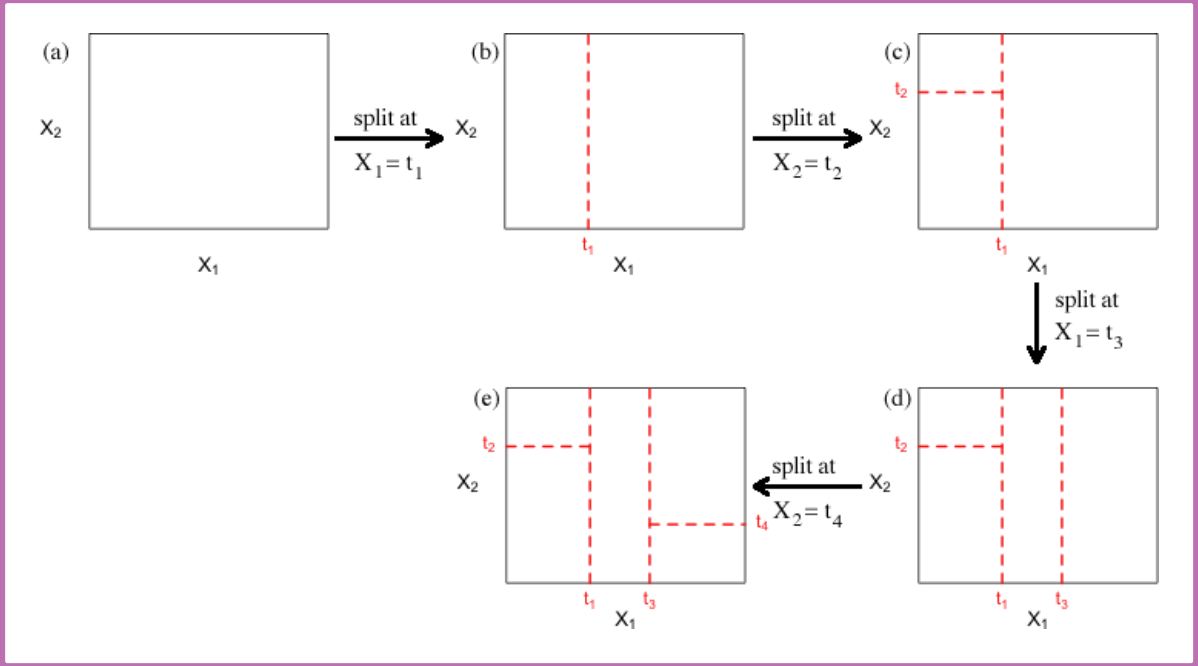
K-fold Cross-Validation

- Since the result of holdout method may be unstable due to randomness of sampling, K-fold cross-validation helps improve over the holdout method.
- Randomly split entire dataset into k folds
- For each iteration of the process, each fold is selected as the test set while remaining k−1 folds as the training set
- Training set is used to train the model and the test set is used to compute the performance (or error) measure.
- Repeat this until each of the k-folds has served as the test set.
- The sum/average of k recorded measures is cross-validated performance.

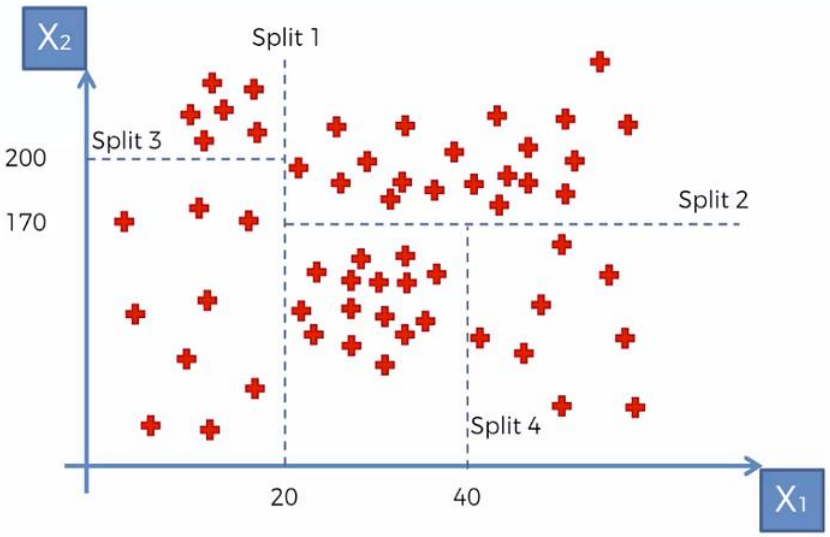




Decision Tree

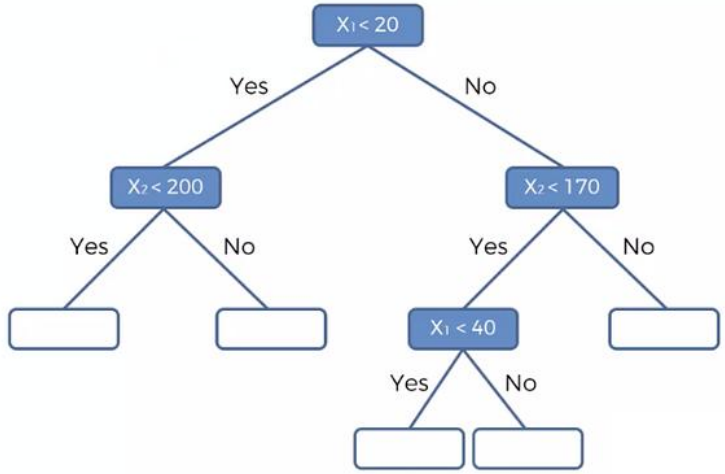


Decision Tree



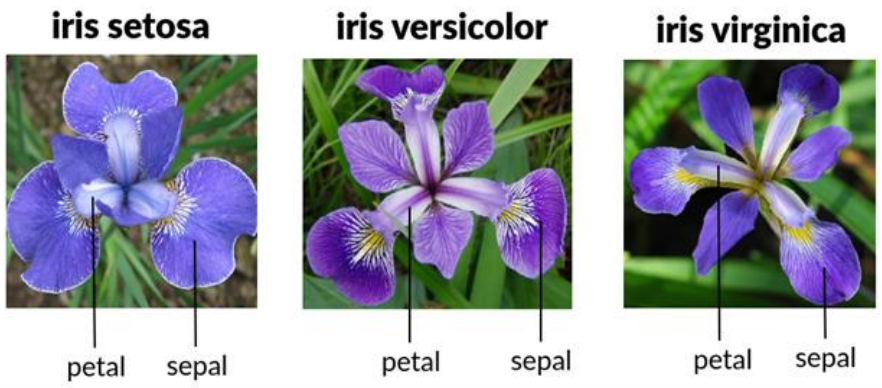
- A supervised method for regression and classification
- Trees spit the input space into a number of simple regions.
- Since the set of splitting rules used to segment the input space can be summarized in a tree, such models are known as decision trees.

- **Types of Decision tree model:**
  - Regression tree (Output variable is continuous)
  - Classification tree (Output variable is nominal or binary)

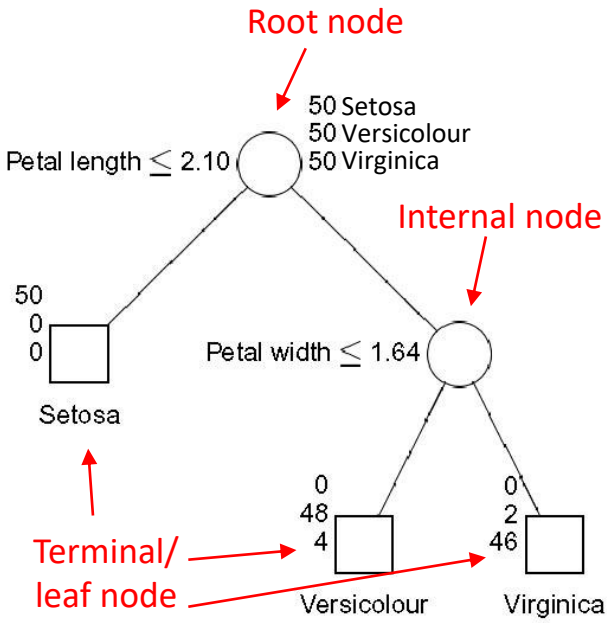


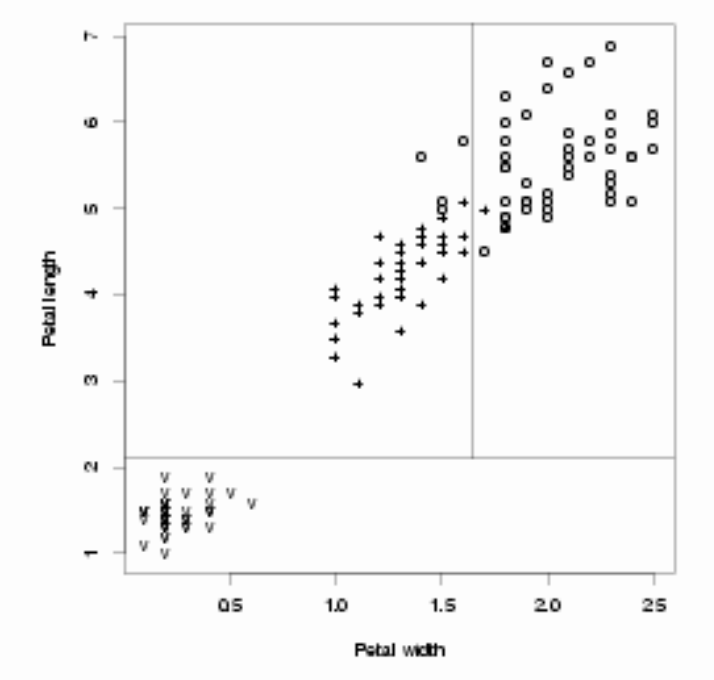
- After forming the tree, predict the output variable by
  - Regression: e.g. Mean of training data within each region (leave)
  - Classification: e.g. Majority vote within each region (leave)

Decision Tree: Example



- **Iris flowers**
  - Output: Species
  - Inputs: petal (花瓣) length and width, sepal (萼片) length and width
  - 3 species; 50 observations per species
- **Objective:** to predict the class of species based on the 4 predictor variables.

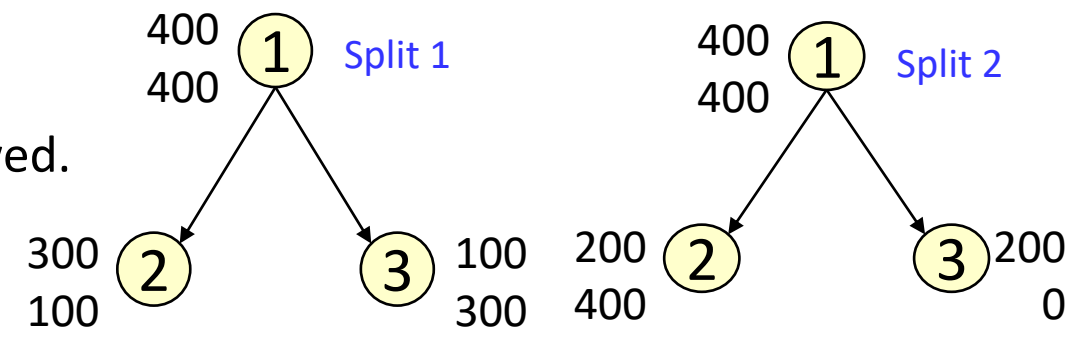




How to Build a Decision Tree?

- **Aim:** to generate more homogeneous leaves
- Typically, binary split (2 branches) is considered.
- In general, multi-way split (3 or more branches) is allowed.

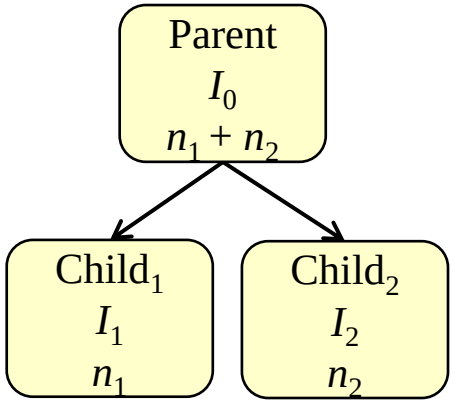
Which split is better?



- **Recursive partitioning algorithm:**
  - Starting from the root node
  - At each node, list out all possible splits (eg. petal length ≤ 2.1?)
  - For each split, compute the information gain:

$$IG = I_0 - \left(\frac{n_1}{n_1 + n_2} I_1 + \frac{n_2}{n_1 + n_2} I_2 \right)$$

    where I_i measures the impurity of the child node i based on training data.
  - The best split of the node is the one with the largest information gain (Why?)
  - Repeat this process to each of the child nodes



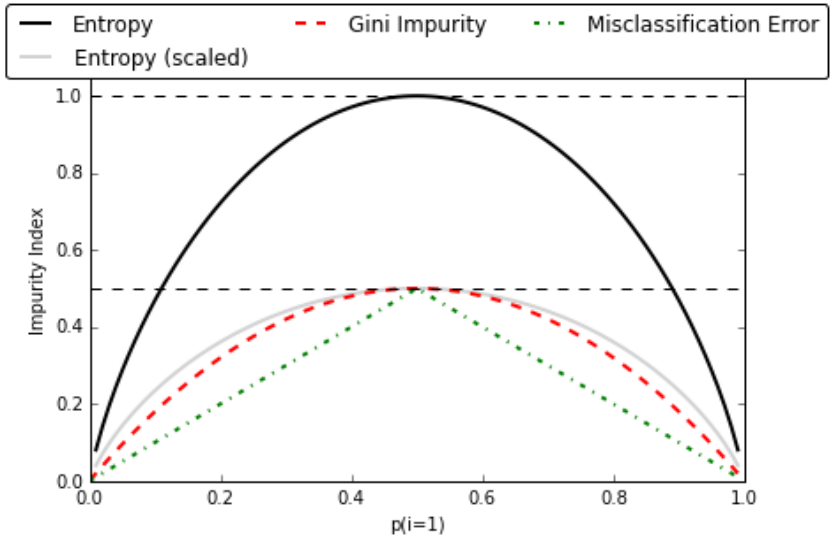
Impurity Function

```
sklearn.tree.DecisionTreeClassifier:
criterion = 'gini' | 'entropy'

.fit (fit tree)
.predict (predict class)
.predict_proba (predict class probability)
```

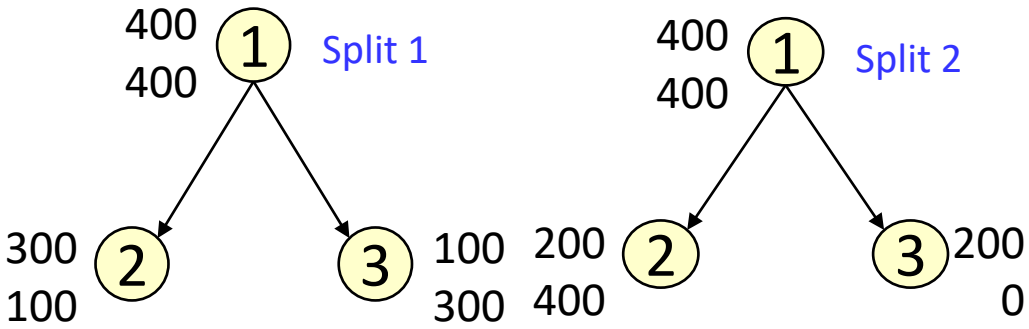
- Choices of Impurity for discrete response:
  - Gini ($G = 1 - \sum_j p_j^2$)
  - Entropy ($E = -\sum_j p_j \log_2 p_j$)
  - Misclassification error ($M = 1 - \max p_j$)

Note: $\log_2(x) = \log(x)/\log(2)$

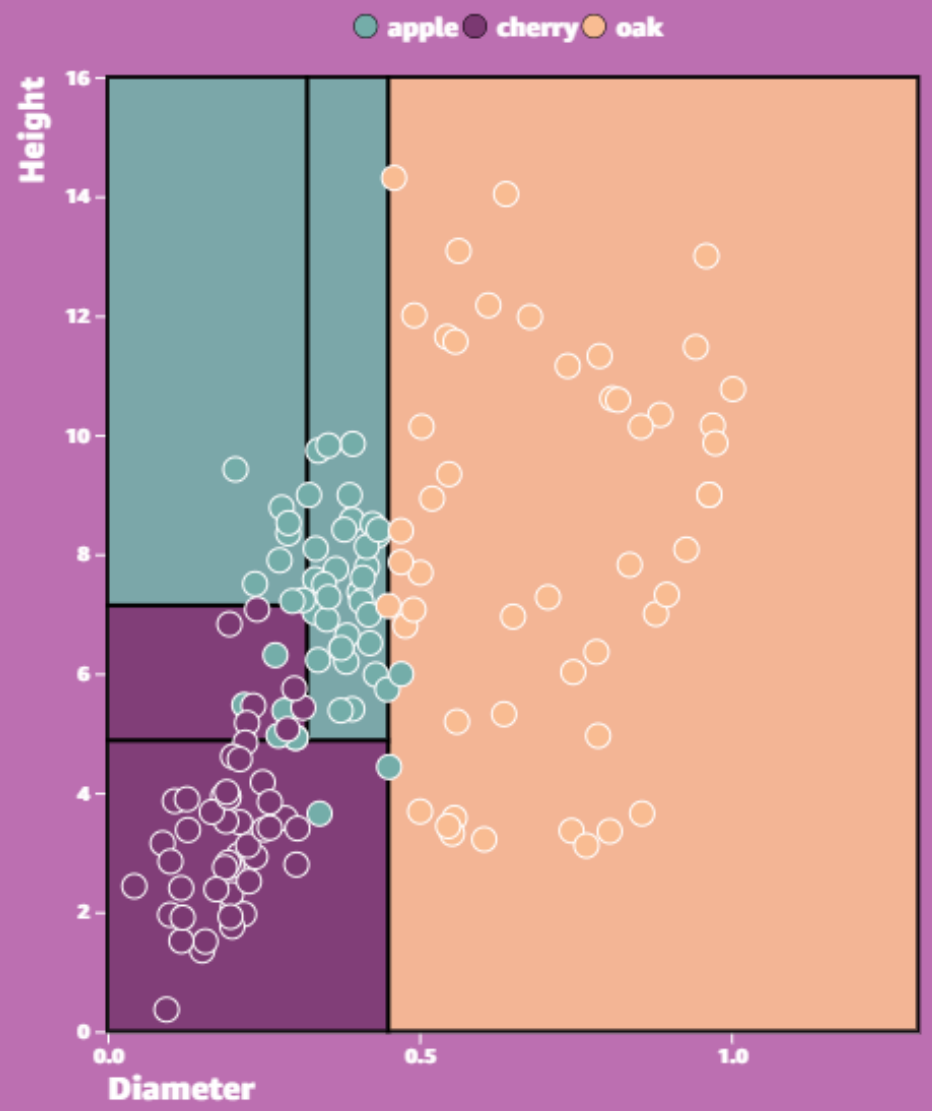


- Choices of Impurity for continuous response:
  - Mean squared error (i.e. variance)
  - Mean absolute error

- Which split is better based on Gini?
  - For split 1's node 1: Gini = $1 - 0.5^2 - 0.5^2 = 0.5$
  - For split 1's nodes 2 and 3: Gini = $1 - 0.75^2 - 0.25^2 = 0.375$
  - For split 1: ΔGini = $0.5 - 0.5(0.375) - 0.5(0.375) = 0.125$
  - For split 2: ΔGini = $0.5 - 0.75(0.444) - 0.25(0) = 0.167$



Decision Tree Demo

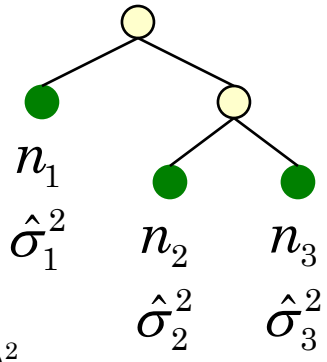


Impurity Function for Continuous Response

```
sklearn.tree.DecisionTreeRegressor:
criterion = 'squared_error'|'absolute_error'

.fit (fit tree)
.predict (predict outcome value)
```

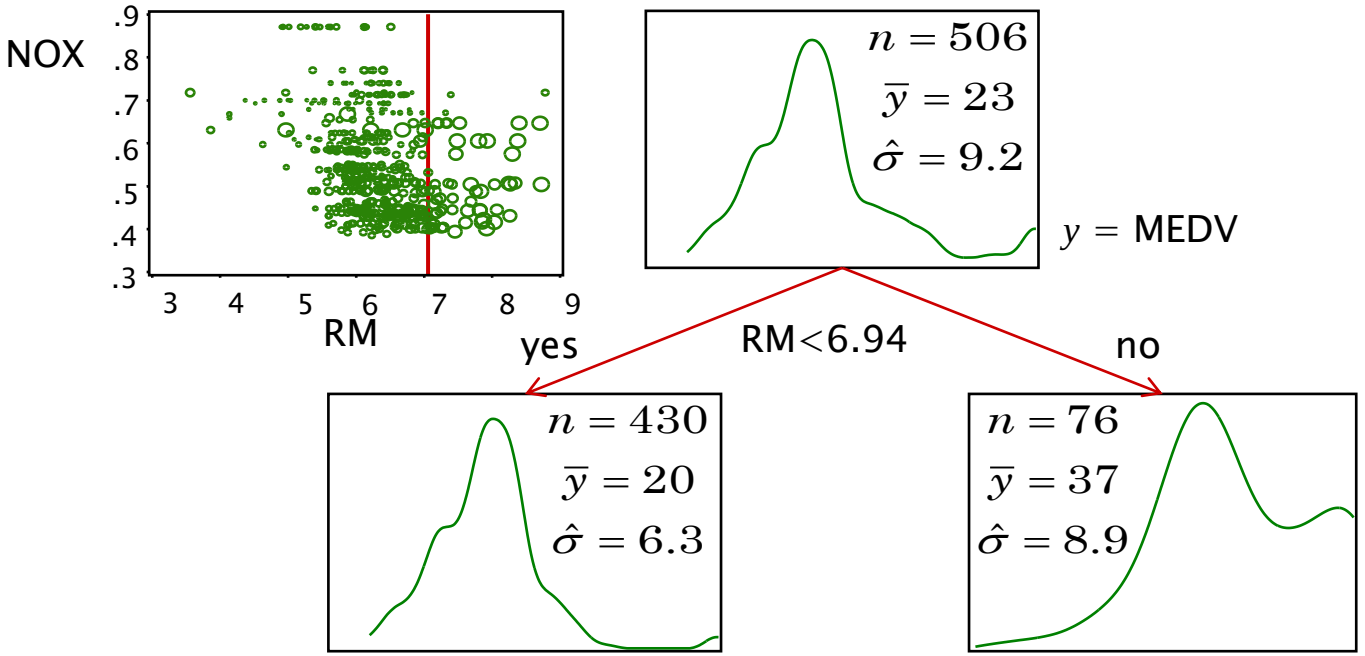
- Based on impurity function of a node
  - Mean squared error (i.e. variance)
  - Mean absolute error



$$\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$$

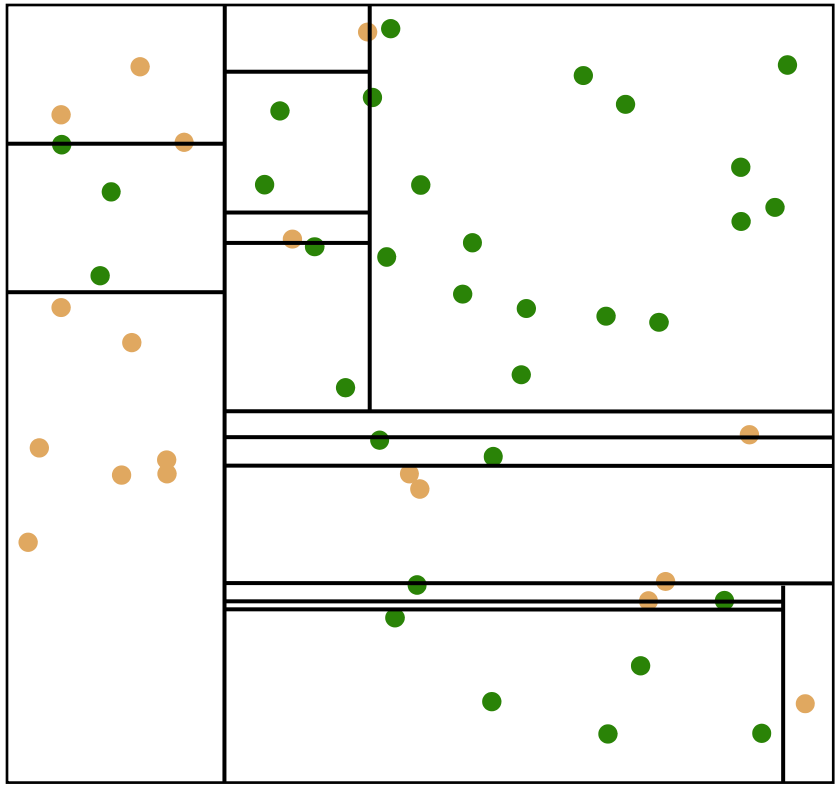
$$=\sum_{t=1}^{T}\frac{n_t}{n}\left(\frac{1}{n_t}\sum_{i=1}^{n_t}(y_{ti}-\bar{y}_t)^2\right)=\sum_{t=1}^{T}\frac{n_t}{n}\hat{\sigma}_t^2$$

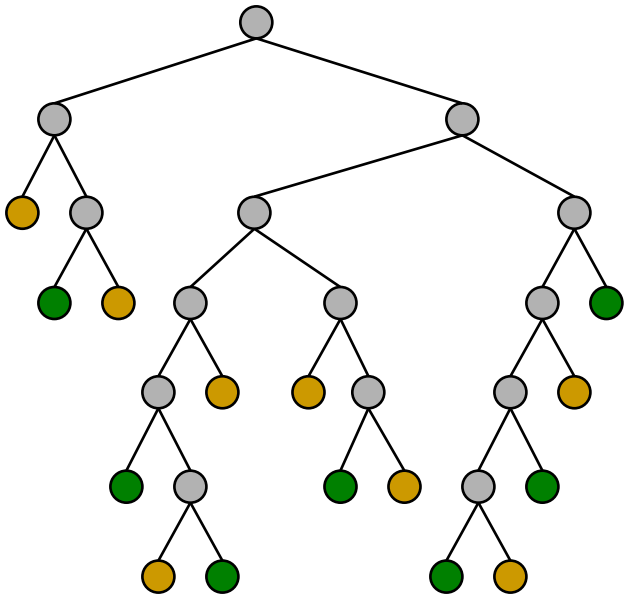
- Variance reduction for Boston Housing Data:



Tree Pruning
(Optimal size of a tree)

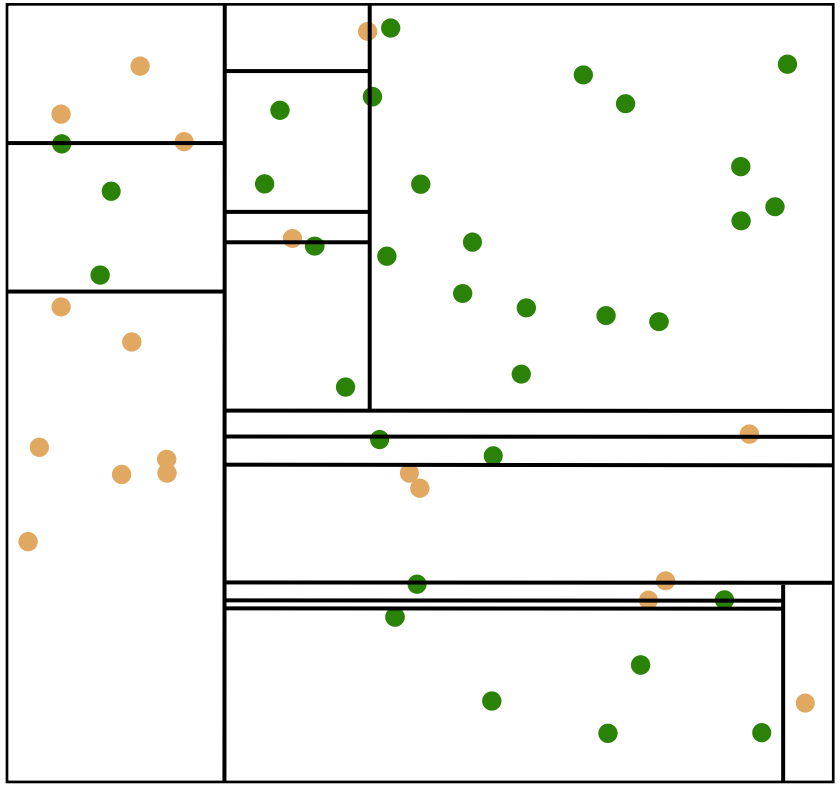
What is the ‘Suitable Size’ of a Tree?



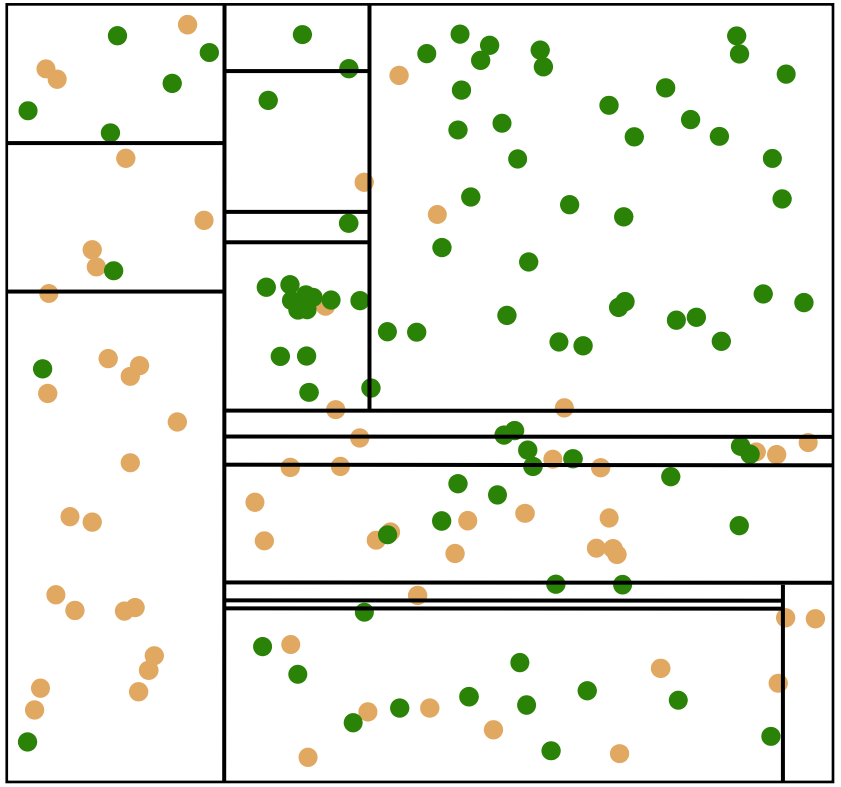


Overfitting

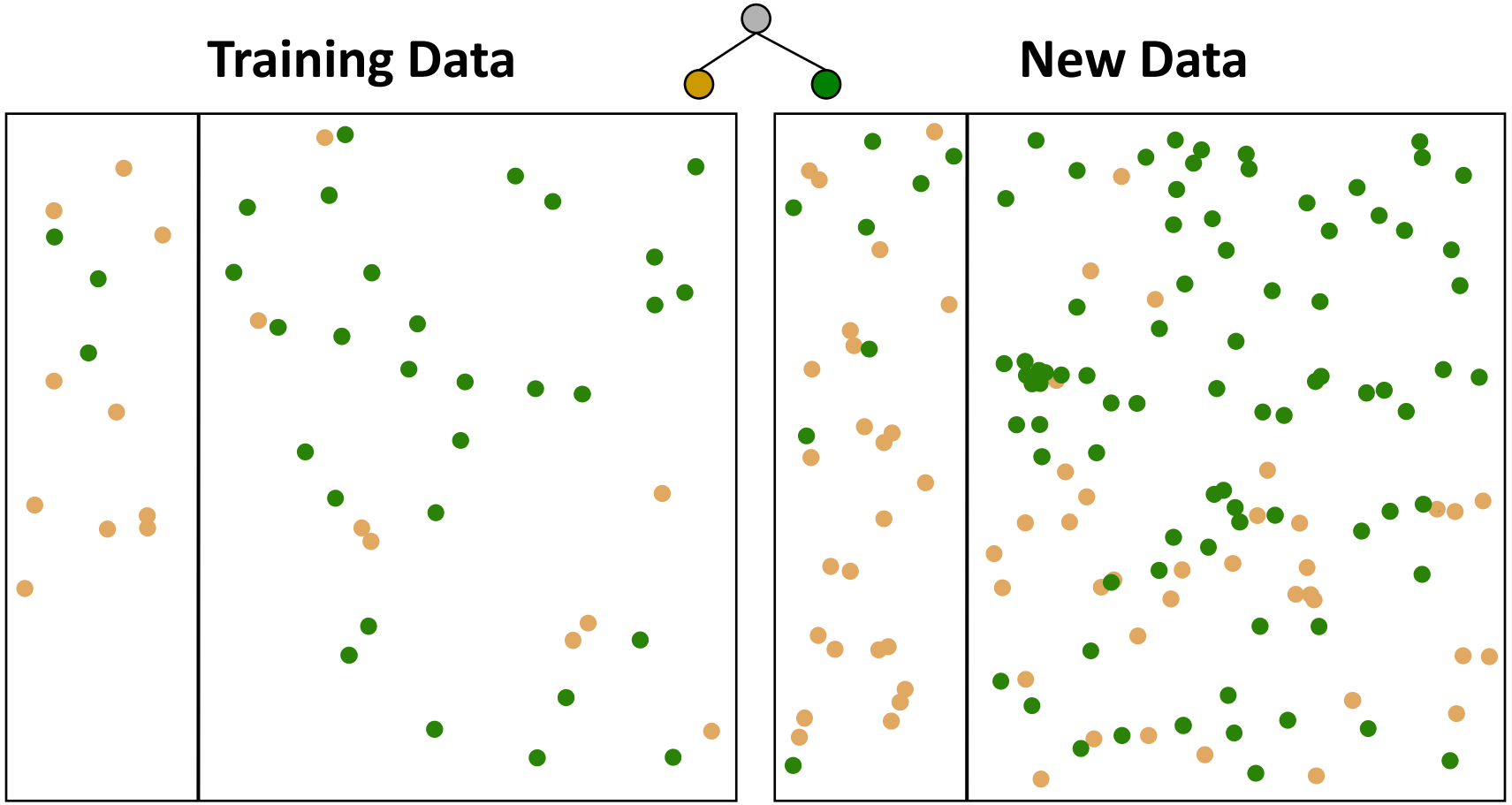
Training Data



New Data



Underfitting



Stop-Splitting Rule

```
sklearn.tree.DecisionTreeClassifier /
sklearn.tree.DecisionTreeRegressor:

max_depth = None,
min_samples_split = 2,
min_samples_leaf = 1,
max_leaf_nodes = None,
ccp_alpha = 0.0
```

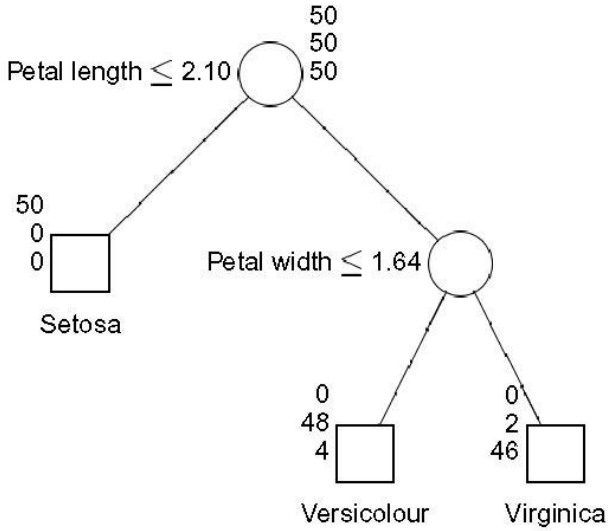
- A simple method:
  - continuous splitting until every node is pure or contains only one observation.
  - fit training data perfectly but may predict poorly on new data.
- Two approaches:
  - Top-down stopping rules (pre-pruning)
    - Set the minimum number of samples required in a leaf node (`min_samples_leaf`)
    - Set the minimum number of samples required to split a node (`min_samples_split`)
    - Limit the depth of the tree (`max_depth`)
  - Bottom-up assessment criteria (post-pruning)
    - Grow a big tree first and remove branches which may not provide significant explanatory power for the output (cost-complexity pruning (`ccp_alpha`))

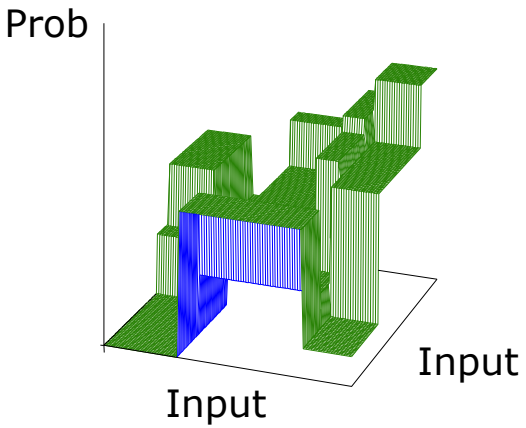
Class Activity 2:

Lab 3: Regression and Decision Tree

Advantages of Decision Tree

- Simple to understand and to interpret. Trees can be visualized.
- Requires little data preparation. Other techniques often require data normalization, dummy variables need to be created and missing data imputation. In sklearn 1.6.1, for each potential split value, `DecisionTreeClassifier` and `DecisionTreeRegressor` will evaluate the split with all the missing values going to the left node or the right node.
- Able to handle both numerical and categorical data.
- Robust to outliers in inputs
- Automatically select input variables and accommodate nonlinearity





Disadvantages of Decision Tree

- Overfitting is one of the most practical difficulty for decision tree models.
  - Solution: pre-pruning and post-pruning
- Most algorithms use univariate splits
  - Solution: Linear combination split ($a_1x_1+a_2x_2< c$?)
- Unstable fitted tree
  - Often a small change in the data result in a very different series of splits
  - Solution: Bagging, Random Forest
- Lack of smoothness (step function) in regression tree
  - Solution: MARS
- Splitting turns continuous input variables into discrete variables.
  - Solution: tree-based regression
- Spitting using a "greedy" algorithm
  - While each split is optimal, the overall tree is not.

Remarks

When important class is rare, use oversampling:

- An effective data augmentation method for numerical data: SMOTE (Synthetic Minority Oversampling TEchnique) to synthesize new data from the minority class.
- A random observation from the minority class is first chosen. Then k of the nearest neighbors for that observation are found (typically k=5). Then, depending on the level of oversampling required, a fraction of them are chosen randomly. For each sampled neighbor is chosen and a synthetic observation is created at a randomly selected point between the two observations in feature space. See https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/

THANK YOU